Regis University CPS
CS361 Programming Assignment 1
due by midnight Sunday of Week 1 (i.e. the last day of week 1)

Program #1

Assume an airport parking service wants to display the sales by parking type in their lot.
The parking type, number of cars, and total collected in one week are:

In the valet parking, 93 cars with total collected of $ 6,696.75
In the covered parking, 532 cars with total collected of $ 23,501.00
In the uncovered parking, 3881 cars with total collected of $ 139,122.20
In the overflow lot, 0 cars with total collected of $ 0.00

Requirements :

Write a program, using output manipulators, to produce the following output **exactly:**

```
Lot Name:          ABC Parking
Week:          Sept 7-13, 2014

   +--------------------------------------+
      Parking     Number         Total
      Type        of Cars        Collected
   +--------------------------------------+
      Valet            93          6696.75
      Covered         532         23501.00
      Uncovered      3881        139122.20
      Overflow          0             0.00
   +--------------------------------------+
```

Output Notes:

- The **Lot Name** and **Week *values*** are aligned on the right.
- The 'P' and 'T' of the "Parking Type" ***column labels*** appears in column 7 of display screen (i.e. there are 6 spaces before the word "Parking" and before the word "Type").
- The **Number of Cars *values*** are aligned on the right and the **Total Collected *values*** are rounded to 2 decimal places and are aligned on the right.

Do not convert the **Number of Cars** and **Total Collected *values*** in the your **.cpp** code to strings (e.g. "93" or "6696.75"). They should both be coded in the output statements as numbers (e.g. 93 and 6696.75).

Minimally, you ***must*** use the **setfill** output manipulator to create and format the dividing lines, and you must use the **setw** and **setprecision** output manipulators to format the numeric output in the table.

No variables or constants should be used in this program.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program #2

The airport parking service noticed that most of their customers park in uncovered parking. Therefore, you will write a program that computes a bill for those customers. The daily rate for uncovered parking is $ 9.25. The hourly rate for uncovered parking is 40 cents an hour. Customers are also charged a tax of 7% of the parking charges, and an airport access fee of $ 2.30 per visit.

Since the number of days parked will vary, the program will need to do some calculations. Prompt the user for the number of hours the customer parked. Use this figure to calculate the total bill.

For every 24 hours, charge the daily rate. Remaining hours will be charged at the hourly rate. (Hint: Use the division operator (/) to determine the number of full days, and use the modulo operator (%) to determine the number of remaining hours over the full days).

Requirements:

The program **must**:

- Define an **integer constant** that holds the number of hours in one day.
- Define four **floating point constants** that will hold:
    - The daily parking rate
    - The hourly parking rate
    - The airport access fee
    - The tax rate
- Define a **string constant** for the parking lot name ("ABC Parking").
- Define a **character constant** for the parking type (this program will currently only be run for customers parking in uncovered parking) that contains 'U' for uncovered parking.
- Define a **string variable** to store a license plate number (e.g., "XXX234").
- Define an **integer variable** to store the number of hours parked.
- Define five **floating point variables** to store parts of the bill including the subtotal and total.
    - The daily charge
    - The hourly charge
    - The parking subtotal
    - The tax charge
    - The total bill
- Define any other variables you decide are necessary for this program.
- Display a program description, followed by **prompts** and **input statements** to read license plate number and number of hours parked from the user.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample Input

```
ABC Parking
Uncovered Parking Billing Program

Enter license plate number: XXX234
Enter number of hours parked: 80
```

- Calculate the daily charges and the hourly charges.
- Add them together to determine the parking subtotal.
- Calculate the taxes on the subtotal.
- Add the subtotal, airport access fee, and taxes to get the total.
- Display a few blank lines to separate the input from the output.
- Use **output statements** to display the results to the screen, in the following format:

Sample Output

```
ABC Parking Receipt

License Plate:            XXX234
Parking Type:                  U
Hours Parked:                 80

Daily Charges              27.75
Hourly Charges              3.20
                          --------
Parking Subtotal           30.95
Taxes                       2.17
Airport Access Fee          2.30
                          --------
Total Parking Bill         35.42
```

NOTE: All values should be aligned right. The dollar figures should be displayed to 2 decimal places and should all line up on the decimal point.

Documentation

1) For each program, be sure to include top of program comments (as specified in course Content section 1.8) that include:
   a) Your name, the course, and the assignment number
   b) What the Program Does including:
      - What is Input
      - Processing/Calculations Performed
      - What is Output

2) Include clarifying comments beside constants and variable declarations

3) Include inline clarifying documentation, as needed.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Submission

This programming assignment is due by **midnight Sunday**.
Your code ***must*** follow the Coding Standards from content section 1.8.

Submit your program source code to the ***Prog Assn 1*** folder in the ***Dropbox*** area of the online course.

Before submitting your program files, you MUST name them as follows:

```
Lastname-wk1-prog1.cpp
Lastname-wk1-prog2.cpp
```

For example:

```
Smith-wk1-prog1.cpp
Smith-wk1-prog2.cpp
```

Both program files should be attached to **one** dropbox submission for programming assignment 1.

To submit your assignment completing the following steps:

1. Click on ***Dropbox*** in the course Navigation bar.
2. Click on the ***Prog Assn 1*** link in the Folder List.
3. Click on the ***Add a File*** button
4. Browse to the **.cpp** source code file for Program 1 on your computer. Select the file and click the ***Open*** button.
5. Click on the ***Upload*** button
6. Repeat steps 3 – 5 to attach to the **.cpp** source code file for Program 2.
7. Type any comments you want to add for your facilitator in the ***Comments*** box.
8. Click on the ***Submit*** button.

You will receive a Confirmation email for the submission.

Pausing Note (for all CS361 programs)

1) When using CS208's ***Bloodshed*** Dev-C++, the program display window closes as soon as a program finishes running. This can be a problem, because the output will disappear before you have time to read it! (Note: This is ***not*** a problem if you have updated to the newer ***Orwell*** Dev-C++, linked in this course).

Students using a PC running the older Bloodshed Dev_C++ includee the following statements before the `return 0;` statement at end of the **main** function:

```
cout << endl;
system ("PAUSE");
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

These statements keep the display window open, so you can see your program output.
Although this method requires more system overhead, it is simple for beginners to use.

NOTE: `system ("PAUSE");`
can be used on **Code::Blocks** and Orwell Dev-C++ too, if you include the following library:

```
#include <cstdlib>
```

But the **pause** statement is not required on either.

2) The above statements are **not portable**. This means that they will not work on **Linux/Unix** systems or on a **Mac**.

If you are using anything other than Dev-C++ on a PC with Windows, you can substitute the following statements to pause your display:

```
// add this line to the function's variable declaration section
char ch;

// place these lines at the point you wish to pause
cout << endl;
cout << "Press [ENTER] to exit" << endl;
cin.get(ch);
```

Grading

The rubric that will be used to grade your program is included on the next pages.

.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 1 Rubric

Program 1: (34 pts)				
Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Documentation	Top of program comments include: Student name, course, and assignment number, along with a brief description that clearly explains what the program will do.	One of the following is missing or incorrect: Student name, course, assignment number, or description, OR description is too vague.	Documentation has multiple problems with incomplete and/or incorrect items.	No comments in program.
Output Implementation	Used all required output format manipulators (setw, setprecision, and setfill). All numeric values were represented correctly in the code (not converted to strings).	Did not use **one** of the required output format manipulators, OR used all three but had a minor errors in manipulator usage. All numeric values were represented correctly in the code.	Did not use **two** of the required output format manipulators, AND/OR hardcoded numeric values as strings AND/OR had major errors using the manipulators.	Did not use any of the required output manipulators -- simply hardcoded entire output using long strings OR had no program output.
Output Display	Output matches program requirements sample exactly.	Output display has one or two minor differences from the program requirements sample.	Output display has many differences from the program requirements sample.	No output is displayed.
C++ Constructs / Readability / Compiles / Miscellaneous	Demonstrates understanding of program structure. Code is organized and easy to follow, with no issues. There is no unnecessary code and code compiles correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code. Code compiles correctly.	Code is difficult to read or has other major issues. One or two small errors prevent the code from compiling.	A multitude of minor errors or one or more major errors prevents the code from compiling.
Program 2: (66 pts)				
Documentation	Top of program comments include: Student name, course, and assignment number, and clear program description with inputs, processing, and outputs listed. Inline comments are included as necessary.	Missing one of the required items or minor necessary inline comments OR program description is vague OR missing some minor details about the input, output, or processing.	Missing two required items or multiple necessary inline comments OR program documentation is incorrect or missing multiple details about the input, output, or processing.	Few or no comments in program.

©2014, Regis University

Constant Usage	All required constants created as specified in requirements, using descriptive, all capitalized names (per coding standards). Only constant names are used in the executable code (no hardcoded constant values).	One or two minor errors in constant declarations, but all required constants created, OR missing one required constants, OR one constant name not used in code (hardcoded value instead).	Missing two or three required constants OR many errors in constant declarations OR some constant names not descriptive OR constants created but seldom used in code (used hardcoded values instead).	None of the constant names were descriptive, OR missing more than three required constants, OR other major errors in constant declarations or usage.
Variable Usage	All required variables created within **main** as specified in requirements, using correct data types and descriptive names beginning with a lowercase letter (per coding standards). Compound words are separated with underscores or capitalization.	All required variables created. But created global variables, instead of local variables OR one or two other minor errors in variable declarations with typing, non-descriptive names, or did not follow coding standard naming conventions.	Missing one or two required variables OR some variable names not descriptive OR many errors in variable declarations.	Few or none of the variable names were descriptive, OR missing more than two required variables, OR major errors in the variable declarations or data types assigned.
Program Input	Displays descriptive prompt(s) indicating what the user needs to provide, and reads the required input correctly.	Reads the required input correctly, but does not display appropriate prompt(s) indicating what data the user needs to provide OR reads unnecessary extra input.	Some kind of prompt issued and data read, but not stored into correct variable, or other issues preventing proper use of input data.	Fails to read required input. Hardcoded values used to obtain the results or no results obtained.
Program Logic / Calculations	The program logic accomplishes all program requirements. Mathematical equations compute the correct results.	One mathematical equation computes incorrect results, or the program logic does not accomplish one of the program requirements.	Multiple mathematical equations compute incorrect results OR program uses input or hardcoded value(s) instead of performing a computation.	Program logic /calculations accomplish few or none of the program requirements.
Program Output	Displays all of the required output, formatted according to program requirements	Displays all the required output, but there are one or two minor errors in the output format.	Does not display some of the required output OR displays all of the required output, but has multiple errors in the output format.	Completely incorrect output display, or does not display any output at all.

©2014, Regis University

C++ Constructs / Readability / Miscellaneous	Demonstrates understanding of program structure. Code is organized and easy to follow, with no issues, and there is no unnecessary code.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is difficult to read OR has other major issues.	The code is unreadable.
Compilation	Code compiles correctly.	One or two minor error prevents the code from compiling.	Multiple minor errors prevent the code from compiling.	One or more major errors or a multitude of minor errors prevent the code from compiling.
Delivery	Submitted on time	Submitted 1-2 days late (2% deducted per day late)	Submitted 3-5 days late (2% deducted per day late)	Not submitted within 5 days of due date

©2014, Regis University